FPT EVENT MANAGEMENT SYSTEM

Project Control Document — OJT Microservices Migration

> **Dự án:** Hệ thống Quản lý Sự kiện FPT University
> **Kiến trúc:** 6 Microservices trên AWS Lambda arm64
> **Trạng thái:** 95% hoàn thiện · 0 Compile Errors · DB 0.84 MB
> **Repository:** AK17-LeonSatoru/FPT_EVENT_MANAGEMENT_80percent-Microservices

---

MỤC LỤC

1. Product Overview
2. Architecture
3. Feature Specification
4. Data Architecture
5. Technical Pipeline
6. API Specification
7. State Machines
8. Development Roadmap
9. Risk Matrix
10. Critical Path
11. Success Criteria
12. Conventions

PHẦN 1: PRODUCT OVERVIEW

1.1 Tầm nhìn

Nền tảng quản lý sự kiện tập trung cho FPT University, hỗ trợ toàn bộ vòng đời sự kiện — từ lúc tổ chức đăng ký địa điểm, mua vé, nhận QR ticket, đến check-in tại sự kiện — trong một hệ thống duy nhất, bảo mật và có khả năng mở rộng. Hệ thống được chuyển đổi từ Modular Monolith sang **6 Lambda Microservices**, áp dụng Saga Pattern để đảm bảo tính toàn vẹn giao dịch trong môi trường phân tán.

1.2 Scope — Giai đoạn Microservices Migration

Aspect	In Scope	Out of Scope
Sự kiện	University events nội bộ FPT	Sự kiện công cộng ngoài campus
Platform	React Web App (mobile-first)	Native mobile app (iOS/Android)
Backend	Go Microservices · 6 AWS Lambda · arm64	Python, Node.js services
Database	MySQL 8.0 trên AWS RDS (Shared DB)	Database-per-service, NoSQL, DynamoDB
Auth Phase 1	Simple JWT + Lambda Authorizer	Amazon Cognito, social login
Payment	VNPay Gateway + Internal Wallet	Stripe, PayPal, POS trực tiếp

Aspect	In Scope	Out of Scope
Inter-service	Direct Lambda Invoke + HTTP REST	gRPC, SQS/SNS message queue
Phân bổ ghế	10×10 matrix · VIP-first algorithm	Custom floor plan, drag-and-drop
Thông báo	SMTP email + Virtual notifications	SMS, push notification native
Check-in	QR code scan	NFC, facial recognition
Báo cáo	Built-in analytics · CSV export	BI dashboard, external OLAP
Data curation	Manual by Admin	Auto-crawl, AI tagging

1.3 Core User Journey — Tổng quan

```
Organizer tạo Event Request (chọn Venue Area + khung giờ + trần giá vé)
↓
Admin duyệt → Event APPROVED → Mở bán vé
↓
User đăng ký vé (chọn Category: VIP / STANDARD)
↓
System phân bổ Seat tự động (VIP rows A–C · STANDARD rows D–J · INSERT IGNORE)
↓
Wallet Saga: RESERVE tiền → CREATE Ticket → CONFIRM hoặc RELEASE (compensation)
↓
Notification Lambda gửi Email + PDF vé + QR Code
↓
Ngày sự kiện: Staff quét QR → Check-in xác thực
↓
Kết thúc: Auto-close · Reports · Hoàn tiền (nếu có) · Venue AVAILABLE
```

Tổng thời gian target: **< 3 phút** từ bấm "Mua vé" đến nhận QR ticket qua email.

1.4 Tech Stack

Layer	Technology	Hosting
Frontend	React 18 + TypeScript 5.2 + Vite 5 + Tailwind CSS	Vercel (CDN global)
Backend	Go 1.24 — `github.com/fpt-event-services`	AWS Lambda arm64 (Container Image)
Containerization	Docker + Docker Compose · Multi-stage Build	Local dev: Alpine ~25 MB · Lambda: ECR image
IaC / Deploy	AWS SAM CLI + AWS CloudFormation	S3 artifact bucket
Database	MySQL 8.0 (`go-sql-driver/mysql v1.9.3`)	AWS RDS db.t3.micro (Free Tier)
Auth	JWT HS256 (`golang-jwt/jwt v5.2.0`) + bcrypt	In-Lambda (Phase 1)

Layer	Technology	Hosting
Tracing	AWS X-Ray SDK Go (`v1.8.5`)	AWS X-Ray Console
Logging	Custom Structured Logger (JSON/color)	AWS CloudWatch Logs
PDF Generation	`jung-kurt/gofpdf v1.16.2`	In-Lambda (no external dep)
QR Code	`skip2/go-qrcode v0.0.0`	In-Lambda
Config Store	AWS SSM Parameter Store	`/fpt-events/{env}/system-config`
Payment	VNPay Gateway (HMAC-SHA512)	External redirect
Media	AWS S3 (`aws-sdk-go-v2/service/s3`)	ap-southeast-1, pre-signed URLs

1.5 Team

Role	Code	Focus
Backend Lead	BE-L	Architecture, Saga Engine, SAM infra, AWS, X-Ray, Code Review
Backend Dev 2	BE-2	Event/Venue module, Data seeding, Collection curation
Backend Dev 3	BE-3	Auth/Ticket module, Saga implementation, VNPay integration
Frontend Dev	FE	Entire React UI — all pages, hooks, API client
Product/Founder	PM	Data curation, Venue mapping, Testing, Demo prep

1.6 Timeline

Phase	Thời gian	Mục tiêu
Phase 0	Tháng 11/2025	Foundation: Architecture design · DB schema · SAM setup
Phase 1	Tháng 12/2025–1/2026	Core: 6 Lambda services · InternalClient · Feature Flags
Phase 2	Tháng 2/2026	Advanced: Saga Pattern · API Composition · X-Ray tracing
Phase 3	Tháng 3/2026	Polish: DB optimization (0.84 MB) · 0 compile errors · Demo
Hiện tại	**March 9, 2026**	**95% complete · OJT Demo Ready**

PHẦN 2: ARCHITECTURE

2.1 Solution Architecture — AWS

```
                    USERS (Mobile Browser / Desktop)
                                |
                    ┌───────────▼────────────┐
                    │      VERCEL CDN        │
                    │    React SPA           │
                    │   • Global edge        │
                    │   • Auto SSL           │
                    │   • CI/CD GitHub       │
```

```
                    │  Cost: $0            │
                    └──────────────────────┘
                                │
                                │ HTTPS
                ┌───────────────▼──────────────────────┐
                │          AWS API GATEWAY             │
                │  Public:   /api/*  (Internet)        │
                │  Internal: /internal/* (VPC only)    │
                │  JWT Lambda Authorizer               │
                └──────────────────────────────────────┘
                                │
                                │
                ┌───────────────▼──────────────────────────┐
                │          VPC 10.0.0.0/16                 │
                │  Private Subnets: 10.0.10.0/24           │
                │                   10.0.11.0/24           │
                │                                          │
                │  ┌──────────┐ ┌──────────┐ ┌──────────┐  │
                │  │  Auth λ  │ │ Event λ  │ │ Ticket λ │  │
                │  │  :auth   │ │  :event  │ │ :ticket  │  │
                │  └──────────┘ └──────────┘ └──────────┘  │
                │                                          │
                │  ┌──────────┐ ┌──────────┐ ┌──────────┐  │
                │  │  Venue λ │ │  Staff λ │ │ Notif λ  │  │
                │  │  :venue  │ │  :staff  │ │  :notif  │  │
                │  └──────────┘ └──────────┘ └──────────┘  │
                │               │Direct Lambda Invoke│       │
                │  ┌────────────▼──────────┐                │
                │  │   RDS MySQL 8.0       │  SSM Parameter │
                │  │   db.t3.micro         │  Store         │
                │  │   fpt_event_mgmt      │                │
                │  │   0.84 MB             │                │
                │  └───────────────────────┘                │
                └──────────────────────────────────────────┘
                      │ Traces                   │ Logs
                ┌─────▼──────┐            ┌──────▼──────────┐
                │  AWS X-Ray │            │  CloudWatch Logs │
                │ Service Map│            │  JSON structured │
                └────────────┘            └─────────────────┘
```

Justification — Quyết định kiến trúc:

Quyết định	Lựa chọn	Lý do
Compute	AWS Lambda arm64	Serverless, pay-per-invoke. arm64 (Graviton2) = 20% cost saving. Go binary cold start ~15 ms.
Database	RDS MySQL db.t3.micro	Free Tier 12 tháng. ACID transactions. Row-level locking cho wallet. MySQL array workaround dùng JSON field.
Frontend	Vercel	React SPA từ GitHub. Auto build, SSL, CDN global. Zero config.
Inter-service	Direct Lambda Invoke	Tránh Internal API GW hop. Latency ~2–5 ms thay vì ~15 ms. Không tốn API GW charge nội bộ.

Quyết định	Lựa chọn	Lý do
Auth Phase 1	Simple JWT trong Lambda	Nhanh nhất. Lambda Authorizer cache 300s. Cognito integrate Phase 2+.
Tracing	AWS X-Ray	Cross-Lambda service map tự động. Annotations + Metadata cho debug.
Config	SSM Parameter Store	Secrets không hardcode. Inject vào Lambda environment khi boot.

Cost Estimate MVP:

Service	Free Tier	MVP Usage	Cost
AWS Lambda	1M requests/mo · 400K GB-s	<500K requests	$0
API Gateway REST	1M calls (12 months)	<500K calls	$0
RDS MySQL db.t3.micro	12 months	<1GB data	$0
S3 + CloudFront	5GB, 50GB transfer	<1GB	$0
SSM Parameter Store	10K requests/mo	<1K	$0
Vercel Hobby	Unlimited	React SPA	$0
Total			**~$0 (Free Tier)**

Architecture Evolution Path:

```
MVP (Now — March 2026):
  Vercel (React SPA) + API Gateway + 6 Lambda arm64 + RDS MySQL + S3
  HTTP Polling cho status check (3s interval)
  Simple JWT + Lambda Authorizer
  Direct Lambda Invoke (InternalClient adaptive)

V1.1 (Tháng 6/2026):
  + API Gateway WebSocket → real-time check-in sync
  + Amazon Cognito → managed auth, Google login
  + Amazon SES → thay SMTP (higher deliverability)
  + CloudWatch dashboards + X-Ray Insights

V2 (Tháng 12/2026+):
  + ECS Fargate → nếu cần persistent connections
  + pgvector / Aurora MySQL → AI-based event recommendations
  + DynamoDB Streams → event-driven notifications
  + Python Lambda → ML scoring cho gợi ý sự kiện
```

2.2 Containerization Strategy — Docker

Tại sao Docker?

Hệ thống sử dụng **Docker Compose** cho local development và **Lambda Container Image** cho production. Cùng một `Dockerfile`, hai target:

Target	Base Image	Size	Dùng cho
`local`	`alpine:3.21`	~25 MB	`docker compose up` local dev
`lambda`	`public.ecr.aws/lambda/provided:al2023`	~30 MB	ECR → AWS Lambda

Multi-stage Build Flow

```
Stage 1: golang:1.24-alpine (Builder)
  CGO_ENABLED=0, GOOS=linux, GOARCH=arm64
  go build -ldflags="-s -w" -tags lambda.norpc
  Output: /app/service (static binary, ~12 MB)
        ↓ COPY --from=builder
Stage 2a: alpine:3.21 (target=local)
  + ca-certificates, tzdata
  ENTRYPOINT ["./service"]
  → IsLocal()=true → HTTP server on $LOCAL_PORT
        ↓ OR
Stage 2b: provided:al2023 (target=lambda)
  COPY → /var/task/bootstrap
  CMD ["bootstrap"]
  → IsLocal()=false → lambda.Start(Handler)
```

Key insight: Binary tự phát hiện môi trường qua `AWS_LAMBDA_FUNCTION_NAME`:

- Unset (Docker local / PowerShell) → khởi động HTTP server (`localserver.Start`)
- Set (Lambda runtime inject tự động) → `lambda.Start(Handler)`

Docker Compose — 9 Container Topology

```
┌──────────────────────────────────────────────────────────┐
│                    fpt-network (bridge)                  │
│                                                          │
│  mysql:3306 ──healthcheck──► auth-service:8081           │
│       └──────────────────► event-service:8082            │
│       └──────────────────► ticket-service:8083           │
│       └──────────────────► venue-service:8084            │
│       └──────────────────► staff-service:8085            │
│       └──────────────────► notification-service:8086     │
│                                  └──► gateway:8080       │
│                                            └──► frontend:3000
└──────────────────────────────────────────────────────────┘

         exposed to host:
         3000 (browser) · 8080 (API) · 3306 (DB tools)
```

MySQL Case Sensitivity Fix

Linux MySQL mặc định phân biệt chữ hoa/thường với tên bảng (`lower_case_table_names=0`). Code Go query `Event`, SQL schema tạo `Event` — nhưng Linux MySQL có thể không nhận nếu thiếu flag. Fix:

```
# docker-compose.yml
mysql:
  command: --lower-case-table-names=1
```

Flag này phải được set **trước khi data volume được khởi tạo**. Nếu đã có volume cũ:

```
docker compose down -v   # xóa volume
docker compose up --build
```

S3 Upload — Zero-Waste Pattern (6 bước)

Thay thế Supabase Storage bằng **AWS S3** với cơ chế upload an toàn:

```
1. Frontend → POST /api/events/upload-banner (validate only, no file)
   Server: kiểm tra định dạng, kích thước (max 5 MB), content-type

2. Server → GeneratePresignedURL(s3Key, 15 phút)
   Trả về: { uploadUrl, s3Key, publicUrl }

3. Frontend → PUT uploadUrl (file bytes, không qua backend)
   S3 nhận file trực tiếp từ browser

4. Frontend submit form với s3Key + publicUrl

5. Server → CommitUpload(s3Key): xác nhận object tồn tại
   Nếu object không tồn tại → reject

6. Server ghi publicUrl vào DB → transaction commit
```

Filename Sanitization (trước khi tạo S3 key):

```go
func SanitizeFilename(name string) string {
    // Bước 1: Normalize Unicode (NFD → tách dấu)
    // Bước 2: Strip dấu (combining marks)
    // Bước 3: Chuyển thường
    // Bước 4: Thay khoảng trắng/dấu bằng dấu gạch ngang
    // Bước 5: Chỉ giữ [a-z0-9._-]
    // Bước 6: Thêm timestamp để tránh collision
    // Kết quả: "Sự Kiện Âm Nhạc.jpg" → "su-kien-am-nhac-1741234567.jpg"
}
```

2.3 Backend Architecture — Go Microservices

Dependency Flow (Clean Architecture)

```
services/{name}-lambda/          ← Entry point, Lambda handler, route registration
  references → handler/          ← HTTP handlers (thin controller)
  references → usecase/          ← Business logic (domain rules)
  references → repository/       ← DB access (interface implementation)
  references → models/           ← Domain entities
  references → common/           ← Shared utilities (NO reference back)

common/                          ← Cross-cutting concerns
  config/feature_flags.go        ← 10 Feature Flags
  utils/internal_client.go       ← Direct Lambda Invoke / HTTP adapter
  utils/service_registry.go      ← Service URL resolution
  xray/tracer.go                 ← AWS X-Ray wrapper
  logger/logger.go               ← JSON structured logger
  jwt/, hash/, validator/        ← Security utilities

☑ Dependency Inversion: usecase depends on repository interface, NOT concrete DB
☑ Feature Flags: swap implementation path at runtime, zero-downtime rollback
```

Project Structure

```
fpt-event-management-system/
├── backend/
│   ├── cmd/
│   │   ├── local-api/main.go     # Unified local server (all 6 services, port 8080)
│   │   └── gateway/main.go       # Local API gateway router
│   │
│   ├── common/                   # Shared utilities (imported by all services)
│   │   ├── config/
│   │   │   ├── feature_flags.go  # ⭐ 10 Feature Flags (env var controlled)
│   │   │   └── system_config.go  # SSM config loader
│   │   ├── db/db.go              # MySQL connection pool
│   │   ├── utils/
│   │   │   ├── internal_client.go   # ⭐ Direct Lambda Invoke / HTTP adapter
│   │   │   └── service_registry.go  # Service URL env var lookup
│   │   ├── jwt/jwt.go            # JWT sign/verify
│   │   ├── hash/password.go      # bcrypt hash/compare
│   │   ├── logger/logger.go      # Structured logger (JSON + color)
│   │   ├── xray/tracer.go        # AWS X-Ray wrapper
│   │   ├── pdf/ticket_pdf.go     # PDF ticket generator
│   │   ├── qrcode/qrcode.go      # QR code generator
│   │   ├── vnpay/vnpay.go        # VNPay HMAC-SHA512
│   │   ├── recaptcha/recaptcha.go
│   │   ├── validator/validator.go
│   │   ├── response/response.go  # HTTP response helpers
│   │   ├── email/email.go        # SMTP email sender
│   │   ├── models/
│   │   │   ├── wallet.go         # Wallet, WalletTransaction models
│   │   │   └── report.go         # Report models
│   │   └── scheduler/            # Background cleanup goroutines
```

```
│   │   │       ├── event_cleanup.go
│   │   │       ├── expired_requests_cleanup.go   # 24h rule ⭐
│   │   │       ├── pending_ticket_cleanup.go
│   │   │       └── venue_release.go
│   │   │
│   ├── services/
│   │   ├── auth-lambda/          # fpt-events-auth-prod
│   │   │   ├── main.go           # Lambda entry (tracer.Configure + router)
│   │   │   ├── handler/          # RegisterHandler, LoginHandler, OTPHandler
│   │   │   ├── usecase/          # AuthUsecase (business rules)
│   │   │   ├── repository/       # UserRepository (MySQL)
│   │   │   └── models/           # User, OTP entities
│   │   │
│   │   ├── authorizer-lambda/    # JWT Lambda Authorizer (API GW)
│   │   │   └── main.go           # Validate JWT → Allow/Deny IAM policy
│   │   │
│   │   ├── event-lambda/         # fpt-events-event-prod
│   │   │   ├── handler/          # EventHandler, RequestHandler, SpeakerHandler
│   │   │   ├── usecase/          # EventUsecase (approval workflow)
│   │   │   ├── repository/       # EventRepository
│   │   │   └── scheduler/        # ExpiredRequestsCleanup (per-service)
│   │   │
│   │   ├── ticket-lambda/        # fpt-events-ticket-prod
│   │   │   ├── handler/          # TicketHandler, WalletHandler, VNPayHandler
│   │   │   ├── usecase/          # TicketUsecase (Saga coordinator ⭐)
│   │   │   ├── repository/       # TicketRepository, WalletRepository
│   │   │   └── scheduler/        # PendingTicketCleanup
│   │   │
│   │   ├── venue-lambda/         # fpt-events-venue-prod
│   │   │   ├── handler/          # VenueHandler, AreaHandler, SeatHandler
│   │   │   ├── usecase/          # VenueUsecase (10×10 seat allocation ⭐)
│   │   │   └── repository/       # VenueRepository, SeatRepository
│   │   │
│   │   ├── staff-lambda/         # fpt-events-staff-prod
│   │   │   ├── handler/          # CheckInHandler, ReportHandler, RefundHandler
│   │   │   ├── usecase/          # StaffUsecase (QR verification)
│   │   │   └── repository/       # AttendanceRepository, ReportRepository
│   │   │
│   │   └── notification-lambda/  # fpt-events-notification-prod
│   │       ├── handler/          # EmailHandler, PDFHandler, QRHandler
│   │       └── main.go
│   │
│   ├── config/system_config.json
│   ├── go.mod                    # module github.com/fpt-event-services · Go 1.24.0
│   ├── main.go                   # Monolith fallback (Feature Flags all false)
│   ├── template.yaml             # AWS SAM template (6 functions + VPC + RDS)
│   └── samconfig.toml            # SAM deploy config (ap-southeast-1)
│
├── frontend/                     # React + TypeScript + Vite + Tailwind CSS
├── Database/
│   └── FPTEventManagement_v5.sql # Schema + seed data
└── run-microservices.ps1         # Local dev startup script (Windows)
```

2.3 Frontend Architecture — React

```
frontend/src/
├── api/
│   ├── client.ts          # Axios wrapper, base URL, JWT header injection
│   ├── auth.ts
│   ├── events.ts
│   ├── tickets.ts
│   ├── venues.ts
│   └── wallet.ts
│
├── components/
│   ├── common/            # Button, Card, Modal, Loading, Layout, Badge
│   ├── event/             # EventCard, EventDetail, RequestForm, SpeakerList
│   ├── ticket/            # SeatMap, TicketCard, PurchaseFlow, QRDisplay
│   ├── wallet/            # BalanceCard, TopupForm, TransactionList
│   ├── checkin/           # QRScanner, AttendeeList, CheckInStatus
│   └── report/            # ReportChart, AttendanceSummary, ExportButton
│
├── pages/                 # Route-level: Home, Events, EventDetail, Purchase,
│                          #   MyTickets, Wallet, CheckIn, Reports, Admin
├── hooks/                 # useAuth, useWallet, useTickets, usePolling
├── types/                 # TypeScript interfaces (Event, Ticket, Wallet...)
├── contexts/              # AuthContext (JWT, user state)
└── utils/                 # Date format, currency VND, QR parser
```

Key tech choices:

- **Vite** — fast HMR, < 300ms rebuild, tree-shaking ≤ 150KB bundle
- **Tailwind CSS** — utility-first, mobile-first responsive, rapid prototyping
- **React Router v6** — nested routing, protected routes (auth guard)
- **Lucide React** — consistent icon set, tree-shakeable
- **React hooks + Context** — state management (không cần Redux cho MVP)

2.4 Inter-Service Communication Strategy

Production (AWS): Direct Lambda Invoke

```
InternalClient.isAWSEnvironment() → true
  ↓
pathToLambdaFunction("/internal/wallet/reserve") → "fpt-events-ticket-prod"
  ↓
AWS SDK v2: lambdasvc.InvokeInput{FunctionName, Payload: APIGatewayProxyRequest}
  ↓
Response: APIGatewayProxyResponse (no HTTP overhead, no API GW charge)

Latency: ~2–5 ms per internal call
```

Local Development: HTTP via InternalClient

```
InternalClient.isAWSEnvironment() → false (no AWS_LAMBDA_FUNCTION_NAME)
  ↓
HTTP POST http://localhost:8080/internal/wallet/reserve
  ↓
Retry: max 3 × exponential backoff (500ms → 1s → 2s)
Timeout: 5s per request
Headers: X-User-Id, X-User-Role, X-Correlation-Id, X-Internal-Call: "true"

Latency: ~5–15 ms (local)
```

Context propagation keys (Go `context.Context`):

Key	Giá trị	Mục đích
`jwt_token`	Bearer token	Tái sử dụng auth qua các internal calls
`user_id`	Integer	Tránh parse JWT lại ở downstream services
`user_role`	String	Authorization check không cần gọi Auth service
`request_id`	UUID	Correlation ID cho log aggregation
`trace_id`	X-Ray trace ID	Ghép cross-Lambda trace trên X-Ray console

---

PHẦN 3: FEATURE SPECIFICATION

PHASE 0: AUTHENTICATION & ACCOUNT MANAGEMENT

Flow — Host/User đăng ký

1. Truy cập trang Register → Nhập email (`@fpt.edu.vn` preferred) + password + display name
2. reCAPTCHA v3 validation (server-side verify với Google)
3. Backend tạo account → `bcrypt` hash password (cost factor 12) → gửi OTP 6 chữ số qua email
4. User nhập OTP → verify → account ACTIVE
5. Login → Backend issue JWT (HS256, exp: 24h) → FE lưu vào `localStorage`
6. Mọi request tiếp theo: `Authorization: Bearer <token>` → Lambda Authorizer validate

Participant Join Flow (không cần account)

1. Nhận link sự kiện từ Organizer
2. Nếu chưa đăng nhập → redirect đến Login
3. Sau login → có thể browse và mua vé trực tiếp

Business Rules

- JWT expire sau 24 giờ; refresh cần login lại (Phase 1)
- OTP 6 chữ số, hết hạn sau 10 phút
- bcrypt cost factor 12 — đủ bảo mật, không quá slow trên Lambda 256MB
- reCAPTCHA trên register + login → chống brute force
- Lambda Authorizer kết quả cache 300 giây → giảm overhead

PHASE 1: EVENT REQUEST & APPROVAL WORKFLOW

Flow

1. Organizer đăng nhập → "Tạo sự kiện mới"
2. Chọn **Venue Area** (từ danh sách các khu vực đã được Admin curation)
3. Điền thông tin: Tên sự kiện, mô tả, ngày giờ, ảnh banner, Speaker info
4. Chọn **Price Tier** cho ticket categories: `VIP` (row A–C) và `STANDARD` (row D–J)
5. System validate venue availability (không overlap với event khác)
6. Submit → Event_Request status = `PENDING`
7. Admin nhận notification → Review → `APPROVE` hoặc `REJECT`
8. Nếu APPROVED → Event status = `OPEN` → open bán vé

Update Deadline Rule — 24h

```
Organizer có thể cập nhật thông tin sự kiện khi status = UPDATING
NHƯNG: Deadline = 24 giờ trước giờ bắt đầu sự kiện

Nếu Organizer không complete trước deadline:
→ ExpiredRequestsCleanup scheduler (chạy mỗi giờ) tự động:
   1. event.status → CLOSED
   2. event_request.status → CANCELLED
   3. venue_area.status → AVAILABLE
   4. Ghi audit log [AUTO_CANCEL]
```

Business Rules

- Trần giá ticket = per person
- Organizer có thể hủy sự kiện khi status `OPEN`, `APPROVED`, hoặc `UPDATING`
- **KHÔNG** hủy được khi đã trong vòng 24h của APPROVED event
- Session expire sau 30 phút nếu không có participant nào join
- Host (Admin) có thể override bất kỳ status nào

PHASE 2: TICKET SALES & SEAT ALLOCATION

Flow

1. User mở Event Detail → xem thông tin, còn bao nhiêu vé
2. Chọn Category: **VIP** hoặc **STANDARD** → Chọn số lượng
3. System chạy Seat Allocation Algorithm → gán seat code tự động
4. Hiển thị seat được gán (A3, B7, ...) → User confirm
5. Chọn phương thức thanh toán: **Wallet** hoặc **VNPay**
6. Proceed to Phase 3 (Payment Processing)

Seat Matrix (10×10)

```
Sơ đồ ghế mỗi Venue Area:

     1    2    3    4    5    6    7    8    9   10
A  [VIP][VIP][VIP][VIP][VIP][VIP][VIP][VIP][VIP][VIP]  ← rows A–C: VIP priority
B  [VIP][VIP][VIP][VIP][VIP][VIP][VIP][VIP][VIP][VIP]
C  [VIP][VIP][VIP][VIP][VIP][VIP][VIP][VIP][VIP][VIP]
D  [STD][STD][STD][STD][STD][STD][STD][STD][STD][STD]  ← rows D–J: STANDARD
E  [STD] ...
...
J  [STD][STD][STD][STD][STD][STD][STD][STD][STD][STD]

Tổng: 100 seats/area · 30 VIP · 70 STANDARD
INSERT IGNORE → tự động resolve concurrent booking conflict
```

Business Rules

- VIP category → **ưu tiên phân bổ từ row A** (VIP đến trước, ngồi trước)
- STANDARD → row D trở đi
- Numeric ordering fix: A1 → A2 → ... → A10 (không phải A1 → A10 → A2)
- Ticket status `PENDING` sau khi tạo; chuyển `CONFIRMED` sau khi payment thành công
- `PendingTicketCleanup` scheduler tự hủy PENDING ticket sau payment window hết hạn
- Refund rate hiện tại: **0.52%** (industry-leading thấp)

PHASE 3: PAYMENT PROCESSING — SAGA TRANSACTION

Flow — Wallet Path (Ưu tiên)

```
1. Ticket Service → POST /internal/wallet/reserve
   { userId, amount, ttlSeconds: 300 }
   → reservationId: UUID (tiền bị lock 5 phút)

2. Seat Allocation Algorithm chạy
   → INSERT IGNORE seats, tạo Ticket records (status = PENDING)

3a. Thành công → POST /internal/wallet/confirm
    { reservationId, referenceId: "ticket_1,ticket_2" }
    → Ticket status = CONFIRMED · INSERT WalletTransaction (type=DEBIT)

3b. Thất bại → POST /internal/wallet/release (compensation)
    { reservationId, reason: "seat_allocation_failed" }
    → Hoàn tiền về ví · Ticket records rolled back
    → User nhận thông báo lỗi
```

Flow — VNPay Path

```
1. Tạo Ticket records (status = PENDING)
2. Redirect User đến VNPay payment page
```

```
  3. VNPay gửi IPN callback → Backend verify HMAC-SHA512 signature
  4. Nếu hợp lệ: UPDATE tickets SET status = CONFIRMED
  5. Nếu timeout hoặc cancel: PendingTicketCleanup dọn dẹp
```

Notification sau thanh toán

```
Payment confirmed → Notification Lambda:
  1. Render PDF ticket (gofpdf): tên, seat code, QR, event info
  2. Generate QR code (go-qrcode): encode ticketId + hash
  3. Send email (SMTP): PDF attachment + event details
```

Business Rules

- Wallet TTL reservation: 5 phút → tự động release nếu crash giữa chừng
- VNPay HMAC-SHA512 verify bắt buộc trên mọi IPN callback
- Không tạo DEBIT WalletTransaction nếu Ticket creation thất bại
- Mọi Wallet operation wrap trong DB transaction (BEGIN → COMMIT/ROLLBACK)

PHASE 4: CHECK-IN & POST-EVENT OPERATIONS

Flow

1. Staff đăng nhập bằng account Staff role
2. Mở Check-in screen → Chọn Event
3. Scan QR code từ tickets của attendee (camera hoặc QR scanner hardware)
4. System verify: ticket hợp lệ + chưa check-in + đúng event
5. UPDATE ticket SET status = `USED`, checked_in_at = NOW()
6. Real-time counter: "127/200 đã điểm danh"
7. Sau event: Admin xem Attendance Report, Revenue Report, Refund Report

Business Rules

- QR encode: `ticketId + HMAC(ticketId, secret)` → chống làm giả
- Mỗi QR chỉ dùng được 1 lần (`UNIQUE` check trên checked_in_at)
- Staff chỉ check-in được event thuộc venue area của mình
- VenueRelease scheduler tự động release venue area sau khi event kết thúc

---

FEATURE A: VIRTUAL NOTIFICATION SYSTEM

Concept

Không có bảng `Notifications` riêng. Tất cả notification được **sinh động từ dữ liệu có sẵn** (`Bill`, `Ticket`), giống như cơ chế cache-on-read.

```
┌──────────────────────────────────────────────────────┐
│ Cách thông thường (Tránh)                            │
│ 1. User mua vé → INSERT Ticket                       │
│ 2. INSERT Notification                               │
│ ❌ Duplicate data · ❌ Sync issues · ❌ Tốn storage   │
└──────────────────────────────────────────────────────┘


┌──────────────────────────────────────────────────────┐
│ Virtual Notifications ☑                              │
│ 1. User mua vé → INSERT Ticket only                  │
│ 2. GET /api/notifications                            │
│ 3. Backend query Bills (payment success/refund)      │
│    + Tickets (check-in events)                       │
│ 4. Transform thành notification format on-the-fly    │
│ ☑ Zero storage waste · ☑ Always in sync              │
└──────────────────────────────────────────────────────┘
```

Email Notifications (SMTP) vẫn gửi thực tế qua Notification Lambda:

- Payment confirmation + PDF attachment
- Event reminder (T-24h)
- Refund processed
- Event cancelled

FEATURE B: VENUE MANAGEMENT & SEAT MATRIX

Concept

Venues được tổ chức thành **Venue Areas** — mỗi area là 1 khu vực vật lý với sơ đồ ghế cố định 10×10. Admin quản lý hoàn toàn. User chọn event → system map tự động với venue area.

- **MVP:** 3–5 Venues, manual curation bởi Admin
- **Mỗi venue:** 2–4 Venue Areas
- **Mỗi area:** 100 seats (10×10), status: `AVAILABLE` / `BOOKED` / `RESERVED`
- VenueRelease scheduler → auto-release area về `AVAILABLE` khi event kết thúc

Post-MVP:

- Google Maps API integration → auto-suggest nearby venues
- Custom seat layout (không còn fixed 10×10)
- GPS auto-detect → suggest collection gần nhất

PHẦN 4: DATA ARCHITECTURE

4.1 System Migration Dimensions — 10 Feature Flags

Hệ thống sử dụng **10 Feature Flags** như 10 "chiều" kiểm soát hành vi của từng tầng kiến trúc. Mỗi flag là binary: `false` = monolith path (cũ) · `true` = microservice path (mới).

#	Flag ID	Tầng	Cực false (Monolith)	Cực true (Microservice)
FF-1	`USE_API_COMPOSITION`	Data	SQL JOIN trực tiếp cross-domain	API Composition — merge in-memory
FF-2	`VENUE_API_ENABLED`	Service	Direct DB query	HTTP call → Venue Lambda
FF-3	`AUTH_API_ENABLED`	Service	Direct DB query	HTTP call → Auth Lambda
FF-4	`TICKET_API_ENABLED`	Service	Direct DB query	HTTP call → Ticket Lambda
FF-5	`EVENT_API_ENABLED`	Service	Direct DB query	HTTP call → Event Lambda
FF-6	`WALLET_SERVICE_ENABLED`	Domain	`Users.wallet_balance` column	Dedicated Wallet + WalletTransaction
FF-7	`SAGA_ENABLED`	Transaction	Local DB transaction	Distributed Saga (Reserve→Confirm→Release)
FF-8	`NOTIFICATION_API_ENABLED`	Service	Local email/PDF call	HTTP call → Notification Lambda
FF-9	`SERVICE_SPECIFIC_SCHEDULER`	Infra	Common/scheduler shared	Per-service goroutine scheduler
FF-10	`SERVICE_SPECIFIC_DB`	Infra	Shared `db.GetDB()` pool	Per-service DB connection init

Trạng thái Production hiện tại (`template.yaml`): Tất cả 10 flags = `"true"` → Full Microservices mode.

4.2 Feature Flag Impact Matrix

Mỗi flag khi bật/tắt tác động đến nhiều chiều hệ thống — tương tự Impact Vectors trong voting system:

```
FF-1: USE_API_COMPOSITION
  true  → { data_isolation: +0.9, query_latency: +0.2, join_capability: -0.9 }
  false → { data_isolation: -0.9, query_latency: -0.2, join_capability: +0.9 }

FF-6: WALLET_SERVICE_ENABLED
  true  → { domain_separation: +0.9, audit_trail: +0.9, migration_complexity: +0.4 }
  false → { domain_separation: -0.8, audit_trail: -0.3 }

FF-7: SAGA_ENABLED
  true  → { transaction_safety: +0.9, compensation_ability: +0.8, latency: +0.3 }
  false → { transaction_safety: -0.5, latency: -0.3, complexity: -0.5 }

FF-9: SERVICE_SPECIFIC_SCHEDULER
  true  → { domain_ownership: +0.8, deployment_independence: +0.9, port_conflict:
+0.1 }
  false → { shared_ownership: +0.3, coordination_cost: +0.6 }
```

Dimension Coverage:

System Property	Primary Flag	Secondary Flags	Coverage
Transaction Safety	FF-7 (SAGA)	FF-6 (WALLET)	☑ Strong
Service Isolation	FF-1 (API_COMP)	FF-2,3,4,5	☑ Strong
Data Domain Separation	FF-6 (WALLET)	FF-1	☑ Strong
Independent Deploy	FF-9 (SCHEDULER)	FF-10 (DB)	☑ Strong
Observability	—	—	⚠ Built-in X-Ray (no dedicated flag)
Network Resilience	—	FF-2..5 (retry)	⚠ Via InternalClient retry logic

4.3 Curated Data Pool

Tương tự "Dish Pool" trong voting system, FPT Event có **Venue & Event Data Pool** được Admin curation thủ công:

Venues & Areas (tương đương Collections):

Venue	Areas	Seats/Area	Tổng seats
FPT Tower HN	3 (Hội trường A, B, C)	100	300
FPT Campus HCM	4 (Hall 1–4)	100	400
FPT Da Nang	2 (Main Hall, Garden)	100	200
Tổng	**~9 areas**	**100**	**~900**

Ticket Categories (tương đương Price Tiers):

Category	Seat Rows	Price Range	Mô tả
VIP	A–C (30 seats)	200k–500k/vé	Ưu tiên vị trí đầu, quà tặng
STANDARD	D–J (70 seats)	50k–200k/vé	Vị trí phổ thông

Effort tagging thủ công:

Dataset	Số lượng	Effort
Venues + floor plans	3–5	4h setup
Venue Areas	9–15	2h cấu hình
Seat initialization (10×10)	~900 seats	Seeder tự động
Event templates/categories	10+	3h định nghĩa
Binary Choices (8 BC + Impact Vectors)	8	4h calibrate
Tổng data preparation		**~15–20 giờ**

4.4 Database Schema — MySQL

```sql
-- ============================
-- REFERENCE DATA (seeded)
-- ============================

CREATE TABLE binary_choices (
    id          VARCHAR(10) PRIMARY KEY,  -- BC-1 .. BC-8
    tier        VARCHAR(20)  NOT NULL,    -- Physics / Palate / Context / Reward
    option_a    VARCHAR(100) NOT NULL,
    option_b    VARCHAR(100) NOT NULL,
    impact_a    JSON NOT NULL,            -- {"soupy": 0.9, "temperature": 0.3, ...}
    impact_b    JSON NOT NULL,
    sort_order  INT NOT NULL
);

CREATE TABLE venues (
    id          VARCHAR(36) PRIMARY KEY DEFAULT (UUID()),
    name        VARCHAR(200) NOT NULL,
    address     TEXT NOT NULL,
    status      VARCHAR(20) DEFAULT 'active',
    created_at  DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE venue_areas (
    id                  VARCHAR(36) PRIMARY KEY DEFAULT (UUID()),
    venue_id            VARCHAR(36) NOT NULL REFERENCES venues(id),
    name                VARCHAR(200) NOT NULL,
    capacity            INT DEFAULT 100,
    price_tier_min      VARCHAR(20),
    status              VARCHAR(20) DEFAULT 'available',  -- available | booked |
reserved
    created_at          DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE seats (
    seat_id     VARCHAR(36) PRIMARY KEY DEFAULT (UUID()),
    area_id     VARCHAR(36) NOT NULL REFERENCES venue_areas(id),
    seat_code   VARCHAR(5)  NOT NULL,   -- A1, B10, J10
    row_no      VARCHAR(2)  NOT NULL,   -- A..J
    col_no      INT         NOT NULL,   -- 1..10
    status      VARCHAR(20) DEFAULT 'available',
    UNIQUE(area_id, seat_code)
);

CREATE INDEX idx_seats_area_status ON seats(area_id, status);

-- ============================
-- REFERENCE DATA (events)
-- ============================

CREATE TABLE events (
    id              VARCHAR(36) PRIMARY KEY DEFAULT (UUID()),
    title           VARCHAR(300) NOT NULL,
```

```sql
    description     TEXT,
    banner_url      TEXT,
    venue_area_id   VARCHAR(36) REFERENCES venue_areas(id),
    start_time      DATETIME NOT NULL,
    end_time        DATETIME NOT NULL,
    status          VARCHAR(20) DEFAULT 'pending',  --
pending|approved|open|updating|closed|done
    created_by      VARCHAR(36) REFERENCES users(id),
    created_at      DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE event_requests (
    id          VARCHAR(36) PRIMARY KEY DEFAULT (UUID()),
    event_id    VARCHAR(36) REFERENCES events(id) ON DELETE CASCADE,
    status      VARCHAR(20) DEFAULT 'pending',  --
pending|approved|rejected|cancelled
    reviewed_by VARCHAR(36) REFERENCES users(id),
    reviewed_at DATETIME,
    note        TEXT,
    created_at  DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE category_tickets (
    id          VARCHAR(36) PRIMARY KEY DEFAULT (UUID()),
    event_id    VARCHAR(36) REFERENCES events(id) ON DELETE CASCADE,
    name        VARCHAR(50)  NOT NULL,   -- VIP | STANDARD
    price       DECIMAL(12,2) NOT NULL,
    quantity    INT NOT NULL,
    remaining   INT NOT NULL,
    row_start   VARCHAR(2),             -- A (VIP) or D (STANDARD)
    row_end     VARCHAR(2)
);

-- ============================
-- TRANSACTIONAL DATA
-- ============================

CREATE TABLE users (
    id              VARCHAR(36) PRIMARY KEY DEFAULT (UUID()),
    email           VARCHAR(255) UNIQUE NOT NULL,
    display_name    VARCHAR(50),
    password_hash   VARCHAR(255) NOT NULL,
    role            VARCHAR(20)  DEFAULT 'user',  -- user | staff | organizer |
admin
    is_active       TINYINT(1)   DEFAULT 0,
    created_at      DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE otp_codes (
    id          VARCHAR(36) PRIMARY KEY DEFAULT (UUID()),
    user_id     VARCHAR(36) REFERENCES users(id) ON DELETE CASCADE,
    code        VARCHAR(6)  NOT NULL,
    expires_at  DATETIME    NOT NULL,
    used        TINYINT(1)  DEFAULT 0,
    created_at  DATETIME DEFAULT CURRENT_TIMESTAMP
```

```sql
);

CREATE TABLE tickets (
    id              VARCHAR(36) PRIMARY KEY DEFAULT (UUID()),
    category_id     VARCHAR(36) REFERENCES category_tickets(id),
    user_id         VARCHAR(36) REFERENCES users(id),
    seat_id         VARCHAR(36) REFERENCES seats(seat_id),
    status          VARCHAR(20) DEFAULT 'pending',  --
pending|confirmed|used|cancelled
    payment_method  VARCHAR(20),     -- wallet | vnpay
    checked_in_at   DATETIME,
    created_at      DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE bills (
    id              VARCHAR(36) PRIMARY KEY DEFAULT (UUID()),
    user_id         VARCHAR(36) REFERENCES users(id),
    ticket_ids      JSON NOT NULL,              -- ["uuid1","uuid2"]
    total_amount    DECIMAL(12,2) NOT NULL,
    payment_method  VARCHAR(20),
    payment_ref     VARCHAR(100),               -- VNPay transaction ID
    status          VARCHAR(20) DEFAULT 'pending',  -- pending|paid|refunded
    created_at      DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- ============================
-- WALLET (Saga-ready)
-- ============================

CREATE TABLE wallet (
    wallet_id   VARCHAR(36)   PRIMARY KEY DEFAULT (UUID()),
    user_id     VARCHAR(36)   UNIQUE NOT NULL REFERENCES users(id),
    balance     DECIMAL(15,2) DEFAULT 0,
    currency    VARCHAR(3)    DEFAULT 'VND',
    status      VARCHAR(20)   DEFAULT 'active'  -- active | frozen | closed
);

CREATE TABLE wallet_transaction (
    transaction_id  VARCHAR(36) PRIMARY KEY DEFAULT (UUID()),
    wallet_id       VARCHAR(36) NOT NULL REFERENCES wallet(wallet_id),
    type            VARCHAR(10) NOT NULL,   -- CREDIT | DEBIT
    amount          DECIMAL(15,2) NOT NULL,
    balance_before  DECIMAL(15,2),
    balance_after   DECIMAL(15,2),
    reference_type  VARCHAR(50),    -- TICKET_PURCHASE | REFUND | TOPUP
    reference_id    VARCHAR(200),   -- ticket_ids hoặc bill_id
    description     TEXT,
    created_at      DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE wallet_reservation (
    reservation_id  VARCHAR(36) PRIMARY KEY,  -- UUID (Saga ID)
    wallet_id       VARCHAR(36) NOT NULL REFERENCES wallet(wallet_id),
    amount          DECIMAL(15,2) NOT NULL,
    expires_at      DATETIME NOT NULL,         -- NOW() + 300s (TTL)
```

```sql
    reference_type  VARCHAR(50),
    created_at      DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- ============================
-- CROWDSOURCE / AUDIT
-- ============================

CREATE TABLE submissions (
    id              VARCHAR(36) PRIMARY KEY DEFAULT (UUID()),
    venue_name      VARCHAR(200) NOT NULL,
    address         TEXT NOT NULL,
    google_maps_url TEXT,
    suggested_area  VARCHAR(200),
    photos          JSON,               -- ["url1","url2"]
    notes           TEXT,
    submitted_by    VARCHAR(100),
    status          VARCHAR(20) DEFAULT 'pending',  -- pending|approved|rejected
    reviewed_at     DATETIME,
    created_at      DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE audit_log (
    id          VARCHAR(36) PRIMARY KEY DEFAULT (UUID()),
    action      VARCHAR(100) NOT NULL,  -- [AUTO_CANCEL], [REFUND], etc.
    entity_type VARCHAR(50),
    entity_id   VARCHAR(36),
    performed_by VARCHAR(36),
    note        TEXT,
    created_at  DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Key indexes
CREATE INDEX idx_events_status          ON events(status) WHERE status != 'done';
CREATE INDEX idx_events_start_time      ON events(start_time);
CREATE INDEX idx_tickets_user           ON tickets(user_id);
CREATE INDEX idx_tickets_status         ON tickets(status);
CREATE INDEX idx_seats_area             ON seats(area_id);
CREATE INDEX idx_wallet_user            ON wallet(user_id);
CREATE INDEX idx_sessions_pin           ON sessions(pin);
```

4.5 Data Volume MVP

Dataset	Số lượng	Effort
Venues	3–5	2h setup
Venue Areas	9–15	2h cấu hình
Seats (auto-seeded via script)	~900–1500	1h seeder
Events (seed cho demo)	10–20	3h
Category Tickets per Event	2 (VIP + STD)	included

Dataset	Số lượng	Effort
Users (test accounts)	~50	1h
Binary Choices + Impact Vectors	8	4h calibrate
Tổng data preparation		**~15–20 giờ**

PHẦN 5: TECHNICAL PIPELINE

5.1 Seat Allocation Algorithm

```
Input:  eventId, categoryId, quantity (số vé muốn mua)
Output: List<seat_code> hoặc error "Không đủ ghế"

Process:
  1. Xác định row range dựa theo category:
     VIP      → row_no IN ('A', 'B', 'C')
     STANDARD → row_no IN ('D', 'E', 'F', 'G', 'H', 'I', 'J')

  2. Query available seats (với numeric ordering fix):
     SELECT seat_id, seat_code, row_no, col_no
     FROM   seats
     WHERE  area_id = ? AND status = 'available'
       AND  row_no IN (target_rows)
     ORDER BY row_no ASC,
              CAST(SUBSTRING(seat_code, 2) AS UNSIGNED) ASC
     LIMIT  quantity

  3. Book seats (concurrency-safe):
     INSERT IGNORE INTO seat_booking (seat_id, ticket_id, booked_at)
     -- INSERT IGNORE: nếu seat đã bị book concurrent → skip, không lỗi

  4. Verify: LEN(successful inserts) == quantity
     if false → release partial, try next batch từ step 2
     if true  → return assigned seat_codes
```

Edge case: Nếu `available < quantity` → return HTTP 409 "Không đủ ghế trống".

5.2 Saga Transaction Pipeline

```
Input:  userId, eventId, categoryId, quantity, paymentMethod
Output: []ticketId (confirmed) HOẶC error + funds returned

--- WALLET PAYMENT PATH ---

Step 1: RESERVE (lock tiền tạm — "giữ chỗ" trong ví)
  POST /internal/wallet/reserve
  {
    "userId":        123,
    "amount":        500000,
```

```
    "referenceType": "TICKET_PURCHASE",
    "referenceId":   "event_1:category_VIP",
    "ttlSeconds":    300    ← tự động hủy sau 5 phút nếu không confirm
  }
  Response → reservationId: "uuid-abc-123"
  DB: UPDATE wallet SET balance -= amount
      INSERT wallet_reservation (reservationId, TTL)

Step 2: ALLOCATE SEATS (Seat Allocation Algorithm)
  if error → goto RELEASE

Step 3: CREATE TICKETS
  INSERT INTO tickets (seat_id, user_id, status=PENDING)
  if error → goto RELEASE

Step 4a: CONFIRM (Happy Path)
  POST /internal/wallet/confirm
  {
    "reservationId": "uuid-abc-123",
    "userId":        123,
    "referenceId":   "ticket_456,ticket_789"  ← cập nhật sau khi tạo vé
  }
  DB: DELETE wallet_reservation
      INSERT wallet_transaction (type=DEBIT, amount, balance_before, balance_after)
      UPDATE tickets SET status = CONFIRMED
  Result: PDF + QR gửi qua email

Step 4b: RELEASE (Compensation — Saga Rollback)
  POST /internal/wallet/release
  {
    "reservationId": "uuid-abc-123",
    "userId":        123,
    "reason":        "ticket_creation_failed"
  }
  DB: DELETE wallet_reservation
      UPDATE wallet SET balance += amount  ← hoàn tiền
      DELETE tickets WHERE status = PENDING
  Không tạo DEBIT WalletTransaction nào
  Result: User nhận lỗi + tiền trả về ví
```

5.3 Concurrent Access Guard — Row-Level Locking

```
// Triết lý: Row-Level Lock là "Spicy Veto" của hệ thống giao dịch
// Nếu user A và user B đều muốn trừ cùng ví → chỉ 1 người thắng

BEGIN TRANSACTION:

  ↓ Lock row này — NO other transaction can modify concurrently
  SELECT balance FROM wallet
  WHERE user_id = ? FOR UPDATE

  IF balance >= required_amount:
    UPDATE wallet
```

```
    SET balance = balance - required_amount
    WHERE user_id = ?

    INSERT INTO wallet_reservation (reservation_id, wallet_id, amount, expires_at)

    COMMIT  ← Release lock
    RETURN {reservationId, success: true}

  ELSE:
    ROLLBACK  ← Release lock
    RETURN InsufficientFundsError


Auto-cleanup: Nếu reservation tồn tại quá expires_at (không có CONFIRM):
  → PendingTicketCleanup scheduler tự động RELEASE (hoàn tiền, hủy vé PENDING)
```

Kết quả: Race condition khi 2 users đồng thời mua vé bằng cùng 1 ví → **Không thể** xảy ra double-spend.

5.4 API Composition — Cross-Service Data Merge

```
Input:  userId (từ JWT context)
Output: Enhanced ticket list với event + venue info
        (không có SQL JOIN cross-domain)

Process (khi USE_API_COMPOSITION = true):

Step 1: Query local DB (Ticket Service owns this data)
  tickets = SELECT * FROM tickets WHERE user_id = ?

Step 2: Extract unique foreign IDs
  eventIds = DISTINCT(tickets.event_id)          -- [e1, e2, e3]
  areaIds  = DISTINCT(tickets.venue_area_id)     -- [a5, a7]

Step 3: Parallel internal calls
  ┌── GET /internal/events/batch?ids=e1,e2,e3   → Event Lambda
  └── GET /internal/venue/areas/batch?ids=a5,a7 → Venue Lambda
  (cả 2 gọi song song, không sequential)

Step 4: In-memory merge (O(n))
  for each ticket in tickets:
    ticket.event = eventData[ticket.event_id]
    ticket.area  = venueData[ticket.venue_area_id]

Step 5: Return unified response JSON

Fallback (USE_API_COMPOSITION = false):
  SQL JOIN trực tiếp — tất cả trong 1 query (monolith mode)
```

5.5 Event-Venue Availability Matching

```
Input:  venueAreaId, startTime, endTime
Output: {available: bool, conflictEvents: [], capacityFit: float}

Process:
  -- Step 1: Time overlap check
  conflicts = SELECT COUNT(*) FROM events
    WHERE venue_area_id = ?
      AND status IN ('approved', 'open', 'updating')
      AND start_time < ? AND end_time > ?   ← nếu overlap → conflict

  if conflicts > 0:
    RETURN {available: false, conflictEvents: [...]}

  -- Step 2: Scoring (cho admin suggest best area)
  area = GET venue_area WHERE id = ?

  capacity_fit  = area.capacity >= requested_attendees ? 1.0 : 0.5
  timing_fit    = isWeekday(startTime) ? 1.0 : 0.8   ← ưu tiên ngày thường
  price_fit     = event.budget_tier == area.price_tier ? 1.0 : 0.5

  score = 0.5 × capacity_fit + 0.3 × timing_fit + 0.2 × price_fit

  RETURN {available: true, score, area}
```

5.6 Auto-Close Cascade (24-Hour Rule)

```
Trigger: ExpiredRequestsCleanup scheduler
         Goroutine + time.Ticker, mỗi 1 HOUR

Query: Tìm events cần tự hủy
  SELECT id, venue_area_id FROM events
  WHERE status IN ('approved', 'updating')
    AND start_time < NOW() + INTERVAL 24 HOUR

For each expired_event:
  BEGIN TRANSACTION:
    1. UPDATE events
       SET status = 'closed'
       WHERE id = expired_event.id

    2. UPDATE event_requests
       SET status = 'cancelled'
       WHERE event_id = expired_event.id

    3. UPDATE venue_areas
       SET status = 'available'
       WHERE id = expired_event.venue_area_id

    4. INSERT INTO audit_log
       (action='[AUTO_CANCEL]', entity_type='event',
        entity_id=expired_event.id, note='24h deadline exceeded')
```

```
    COMMIT
    → If ANY step fails: ROLLBACK → retry next hour cycle

  Log output: "[AUTO_CANCEL] Event {id} closed - 24h deadline exceeded"
```

5.7 Full Technical Pipeline Diagram

```
[User: "Mua vé"]
        │
        ▼
[Auth: JWT verify via Lambda Authorizer (cache 300s)]
        │
        ▼
[Ticket Lambda — Saga Coordinator]
        │
        ├──────────────────────────────────────┐
        │                                      │
        ▼                                      ▼
[Step 1: POST /internal/wallet/reserve]   [Seat Allocation Algorithm]
[Row-Level Lock → reservationId]          [VIP rows A-C / STD rows D-J]
        │                                  [INSERT IGNORE → seat_codes]
        │                                      │
        └──────────────────────────────────────┘
                        │
                        ▼
             [Step 3: INSERT Tickets (PENDING)]
                        │
          ┌─────────────┴──────────────┐
          │ Success                    │ Failure
          ▼                            ▼
[Step 4a: CONFIRM wallet]    [Step 4b: RELEASE wallet]
[INSERT WalletTransaction]   [Funds returned]
[UPDATE tickets → CONFIRMED] [Tickets deleted]
          │
          ▼
[Notification Lambda]
[PDF ticket (gofpdf)]
[QR code (go-qrcode)]
[SMTP email → User]
          │
          ▼
[Ngày sự kiện: Staff check-in]
[QR scan → verify HMAC → mark USED]
          │
          ▼
[Event ends: VenueRelease scheduler]
[venue_area.status → AVAILABLE]
[Generate Report]
          │
          ▼
[Google Maps → Venue Location 🗺]
```

PHẦN 6: API SPECIFICATION

6.1 Endpoints

```
BASE: /api

AUTH
POST   /auth/register           Đăng ký tài khoản mới
POST   /auth/verify-otp         Xác minh OTP (activate account)
POST   /auth/login              Đăng nhập → JWT
PUT    /auth/profile            Cập nhật thông tin [Auth: User]

EVENTS
POST   /events/request          Tạo Event Request [Auth: Organizer]
GET    /events                  Danh sách Events [Public]
GET    /events/{id}             Chi tiết Event [Public]
PUT    /events/{id}/request     Cập nhật Event Request [Auth: Organizer]
DELETE /events/{id}             Hủy Event [Auth: Organizer/Admin]
POST   /events/{id}/approve     Duyệt Event [Auth: Admin]
POST   /events/{id}/reject      Từ chối Event [Auth: Admin]
GET    /events/{id}/status      Poll trạng thái và slot còn lại [Public]

TICKETS
GET    /events/{id}/categories  Thông tin ticket categories [Public]
POST   /tickets/purchase        Mua vé (trigger Saga) [Auth: User]
GET    /tickets/my              Vé của tôi [Auth: User]
POST   /tickets/{id}/cancel     Hủy vé + refund [Auth: User]
GET    /tickets/{id}/qr         QR code của vé [Auth: User]

VENUES
GET    /venues                  Danh sách Venues [Auth: Admin/Staff]
GET    /venues/{id}/areas       Venue Areas [Public]
GET    /venues/{id}/areas/{aid}/seats  Sơ đồ ghế [Auth: Staff/Admin]

WALLET
GET    /wallet/balance          Số dư ví [Auth: User]
POST   /wallet/topup            Nạp tiền → VNPay redirect [Auth: User]
GET    /wallet/history          Lịch sử giao dịch [Auth: User]
POST   /wallet/vnpay-callback   VNPay IPN callback [Public, HMAC verify]

NOTIFICATIONS
GET    /notifications           Virtual notifications [Auth: User]

CHECKIN (Staff)
POST   /staff/checkin           Check-in bằng QR scan [Auth: Staff]
GET    /staff/events/{id}/attendees   Danh sách điểm danh [Auth: Staff]

REPORTS
GET    /reports/events/{id}     Báo cáo sự kiện [Auth: Staff/Admin]
GET    /reports/users           Báo cáo users [Auth: Admin]
GET    /reports/system          Báo cáo hệ thống [Auth: Admin]
GET    /reports/export/{id}     Export CSV [Auth: Staff/Admin]

ADMIN
```

```
GET    /admin/users              Quản lý users [Auth: Admin]
PUT    /admin/events/{id}/status   Override event status [Auth: Admin]
GET    /admin/submissions        Crowdsource review queue [Auth: Admin]
POST   /admin/submissions/{id}/review   Approve/Reject venue [Auth: Admin]

INTERNAL (VPC-private only — Direct Lambda Invoke)
GET    /internal/user/{id}                  Auth Lambda: user info lookup
GET    /internal/events/{id}                Event Lambda: event detail
GET    /internal/events/batch               Event Lambda: batch lookup
GET    /internal/venue/areas/{id}           Venue Lambda: area detail
GET    /internal/venue/areas/batch          Venue Lambda: batch lookup
POST   /internal/wallet/reserve             Ticket Lambda: Saga Step 1
POST   /internal/wallet/confirm             Ticket Lambda: Saga Step 4a
POST   /internal/wallet/release             Ticket Lambda: Saga Step 4b
POST   /internal/notify/email               Notification Lambda: gửi email
POST   /internal/notify/pdf                 Notification Lambda: tạo PDF
GET    /internal/scheduler/event-cleanup    Event Lambda: trigger manual
GET    /internal/scheduler/expired-requests Event Lambda: trigger manual
GET    /internal/scheduler/venue-release    Venue Lambda: trigger manual
POST   /internal/tickets/{id}/checkin       Ticket Lambda: mark USED
```

6.2 Key Request/Response Examples

POST /events/request — Tạo sự kiện

```
// Request [Auth: Organizer JWT]
{
  "title":        "FPT Tech Talk — Go Microservices",
  "description":  "Workshop dành cho sinh viên công nghệ",
  "venueAreaId":  "uuid-area-1",
  "startTime":    "2026-04-15T09:00:00+07:00",
  "endTime":      "2026-04-15T12:00:00+07:00",
  "categories": [
    { "name": "VIP",      "price": 200000, "quantity": 30 },
    { "name": "STANDARD", "price": 50000,  "quantity": 70 }
  ]
}

// Response 201
{
  "eventId":       "uuid-event-xyz",
  "title":         "FPT Tech Talk — Go Microservices",
  "status":        "pending",
  "requestId":     "uuid-request-abc",
  "venueAreaName": "Hall A — FPT Tower HN",
  "startTime":     "2026-04-15T09:00:00+07:00"
}
```

POST /tickets/purchase — Mua vé (Wallet Saga)

```
// Request [Auth: User JWT]
{
  "eventId":       "uuid-event-xyz",
  "categoryId":    "uuid-cat-vip",
  "quantity":      2,
  "paymentMethod": "wallet"
}

// Response 201
{
  "tickets": [
    { "id": "uuid-t1", "seatCode": "A3", "category": "VIP", "status": "confirmed" },
    { "id": "uuid-t2", "seatCode": "A4", "category": "VIP", "status": "confirmed" }
  ],
  "bill": {
    "id":           "uuid-bill-1",
    "totalAmount":  400000,
    "paymentMethod":"wallet",
    "status":       "paid"
  },
  "message": "Vé đã được xác nhận. PDF ticket đã gửi qua email."
}
```

GET /events/{id}/status — Poll trạng thái sự kiện (3s interval)

```
// Response 200
{
  "eventId":          "uuid-event-xyz",
  "status":           "open",            //
pending|approved|open|updating|closed|done
  "title":            "FPT Tech Talk — Go Microservices",
  "ticketCategories": [
    { "name": "VIP",      "remaining": 28, "total": 30 },
    { "name": "STANDARD", "remaining": 45, "total": 70 }
  ],
  "startTime":        "2026-04-15T09:00:00+07:00",
  "isHost":           false    // true nếu JWT là organizer của event này
}
```

POST /internal/wallet/reserve — Saga Step 1 (Internal)

```
// Request [Internal call — X-Internal-Call: "true"]
{
  "userId":        "uuid-user-123",
  "amount":        400000,
  "referenceType": "TICKET_PURCHASE",
  "referenceId":   "event_xyz:category_vip",
  "ttlSeconds":    300
}
```

```
// Response 200
{
  "reservationId": "uuid-reservation-abc",
  "walletId":      "uuid-wallet-999",
  "amountReserved":400000,
  "expiresAt":     "2026-03-09T10:28:45+07:00",
  "balanceAfter":  100000
}
```

POST /staff/checkin — Check-in bằng QR

```
// Request [Auth: Staff JWT]
{
  "qrData":  "uuid-t1:HMAC_SIGNATURE_HERE",
  "eventId": "uuid-event-xyz"
}

// Response 200
{
  "ticketId":    "uuid-t1",
  "status":      "checked_in",
  "attendee": {
    "displayName": "Nguyen Van A",
    "seatCode":    "A3",
    "category":    "VIP"
  },
  "checkedInAt":  "2026-04-15T08:47:12+07:00",
  "totalCheckedIn": 127,
  "totalAttendees": 200
}
```

PHẦN 7: STATE MACHINES

Event State Machine

```
Organizer creates request
        │
        ▼
    PENDING          Admin reviewing Event_Request
        │
   ┌────┴────┐
   │         │
   ▼         ▼
APPROVED   REJECTED   Admin quyết định
   │
   ▼
  OPEN              Bán vé, participants đăng ký
   │
   ▼
UPDATING            Organizer chỉnh sửa thông tin
```

```
    │
    │ (nếu hoàn thành trước 24h start_time)
    ▼
  OPEN              (quay lại OPEN)
    │
    │ (nếu quá 24h deadline)
    ▼
 CLOSED             [AUTO_CANCEL] scheduler kích hoạt
    │
    │ (sau khi event kết thúc)
    ▼
  DONE              Reports available · Venue AVAILABLE

Any state except DONE:
  Organizer/Admin → CANCEL → CLOSED → venue_area.status = AVAILABLE
  WAITING timeout 30 min no join → auto-expire (PENDING only)
```

Ticket & Payment State Machine

```
User chọn vé + payment
        │
        ▼
    PENDING           Saga running (Reserve → Create → ...)
        │
   ┌────┴──────────────────┐
   │                       │
   ▼                       ▼
CONFIRMED              CANCELLED    Saga compensation / user cancel
   │                       │
   │ (Ngày sự kiện)        │ (nếu có refund) → CREDIT WalletTransaction
   ▼
  USED                 QR đã scan thành công
                       checked_in_at = NOW()

PendingTicketCleanup scheduler:
  PENDING tickets quá payment window → CANCELLED
  → Saga RELEASE → Wallet hoàn tiền (nếu wallet reservation chưa expire)
```

PHẦN 8: DEVELOPMENT ROADMAP

8.1 Phase 1: Microservices Core Infrastructure ☑ (Completed)

~2 tháng | Dec 2025 – Jan 2026

Week	Focus	Tasks
1	Foundation	Repo setup · Go module · SAM template · AWS infra (VPC, RDS, S3) · CI/CD GitHub Actions · `docker-compose.yml` local · DB schema v1

Week	Focus	Tasks
2	Common Package	`InternalClient` · `feature_flags.go` · `logger` · `xray/tracer` · `jwt` · `hash` · `validator` · `response`
3	Auth Lambda	Register · Login · OTP · JWT issue · bcrypt · reCAPTCHA · Lambda Authorizer
4	Venue Lambda	Venue CRUD · Area management · Seat seeder (10×10) · Availability check · VenueRelease scheduler
5	Event Lambda	Event CRUD · Request workflow · Approval · Speaker management · ExpiredRequestsCleanup scheduler
6	Ticket Lambda core	Seat Allocation Algorithm · INSERT IGNORE pattern · PENDING ticket
7	Wallet + Saga	Wallet extraction từ Users · Reserve/Confirm/Release endpoints · Row-level locking · WalletTransaction
8	Ticket Lambda Saga	Saga coordinator · VNPay integration · HMAC-SHA512 verify · PDF + QR generation

Deliverable: 5 Lambda functions (auth, event, ticket, venue, authorizer) working locally via `go run cmd/local-api/main.go`

8.2 Phase 2: Integration & Production Polish ☑ (95% Complete)

~6 tuần | Feb – March 2026

Week	Focus	Tasks
1	Notification Lambda	Email SMTP · PDF ticket · QR Code · Virtual notifications endpoint
2	Staff Lambda	Check-in QR verify · Attendance tracking · Refund processing · Report generation
3	API Composition	`USE_API_COMPOSITION` flag · Batch internal calls · In-memory merge · Feature flag routing
4	AWS X-Ray	`tracer.Configure` trên mỗi service · BeginSubsegment cho DB ops · Correlation ID propagation
5	DB Optimization	`OPTIMIZE TABLE` · Index rebuild · Remove redundant columns · **0.84 MB achieved**
6	Demo Prep	0 compile errors · `sam validate --lint` · E2E test · Demo script · Staging deploy

Deliverable Phase 2: ☑ 0 errors · ☑ 0.84 MB DB · ☑ 6 Lambdas · ☑ AWS SAM deploy ready

8.3 Phase 3: Production Hardening (→ April 2026)

Priority	Tasks
1. Load Testing	Persona simulation: 50+ concurrent ticket purchases · Saga stress test
2. Auth Upgrade	Amazon Cognito integration (Phase 2 auth) · Social login
3. Real-time	API Gateway WebSocket → live check-in counter · Instant seat availability
4. Monitoring	CloudWatch dashboards · X-Ray Insights · SLO alerts
5. Presentation	Demo script · Backup video · Architecture walkthrough

8.4 Scope Escape Valves (cắt nếu hết thời gian Phase 2)

```
Ưu tiên CẮT nếu hết thời gian:
  ❌ Crowdsource venue submission → mock data thủ công
  ❌ Admin review queue UI       → dùng DB direct / Postman
  ❌ Session history page         → không demo
  ❌ S3 photo upload              → hardcode Supabase URLs
  ❌ Advanced report charts       → text summary thay vì chart

KHÔNG ĐƯỢC CẮT:
  ☑ Auth (register + login + JWT)
  ☑ Event Request → Approval workflow
  ☑ Ticket purchase (Wallet Saga path)
  ☑ Seat Allocation (10×10 VIP-first)
  ☑ PDF ticket + QR code gửi email
  ☑ Check-in bằng QR
  ☑ Ít nhất 2 Venue Areas với đủ seat data
  ☑ 10 Feature Flags hoạt động đúng
  ☑ AWS SAM deploy lên staging
```

PHẦN 9: RISK MATRIX

Risk	Prob.	Impact	Mitigation
Saga compensation sai → mất tiền user	Low	Critical	Unit test Reserve/Confirm/Release · TTL auto-release · Wallet balance audit trail
Lambda cold start quá chậm → UX kém	Low	Medium	arm64 Go binary ~15ms cold start · Provisioned Concurrency nếu cần
Seat double-booking khi concurrent	Low	High	INSERT IGNORE + transaction · Load test 50 concurrent users
DB connection pool exhausted	Medium	High	`SERVICE_SPECIFIC_DB` flag · connection pool tuning (max 10/service)
JWT secret leak	Low	Critical	SSM Parameter Store · `NoEcho: true` trong SAM · Rotate nếu compromise
VNPay HMAC bypass	Low	Critical	Verify signature trên MỌI callback · Log IP nguồn

Risk	Prob.	Impact	Mitigation
Feature Flag production accident	Medium	Medium	Rollback: `aws lambda update-function-configuration` < 10s · Default = false
InternalClient timeout → cascade failure	Medium	High	Retry 3× exponential · Circuit breaker (Post-MVP) · 5s timeout per hop
Venue area không đủ data → no match	Medium	High	Seeder bắt buộc ≥ 100 seats/area · Warn nếu remaining < 5%
BE junior chậm hơn estimate	High	Medium	BE-L buffer 20% cho review + unblock · Pair programming key flows
DB 0.84 MB tăng lên khi demo	Low	Low	RDS Free Tier 20GB · Cleanup test data trước demo

PHẦN 10: CRITICAL PATH

```
DB-01 MySQL Schema design
  → DB-02 RDS setup on AWS (ap-southeast-1)
    → BE-01 common/ package (db, jwt, logger, internal_client, xray)
      → BE-02 Feature Flags (feature_flags.go)
        → BE-03 Auth Lambda (register, login, JWT, OTP)
          → BE-04 Lambda Authorizer (JWT verify → IAM policy)
            → BE-05 Venue Lambda (area, seat seeder 10×10)
              → BE-06 Event Lambda (CRUD, approval workflow)
                → BE-07 Ticket Lambda — Seat Allocation Algorithm
                  → BE-08 Wallet Saga (Reserve → Confirm → Release)
                    → BE-09 VNPay integration (HMAC-SHA512)
                      → BE-10 Notification Lambda (PDF + QR + SMTP)
                        → BE-11 Staff Lambda (Check-in QR verify)
                          → INT-01 InternalClient adaptive (Lambda Invoke / HTTP)
                            → INT-02 API Composition (cross-service merge)
                              → INT-03 X-Ray tracing (all 6 services)
                                → INT-04 DB Optimization (0.84 MB)
                                  → INT-05 AWS SAM deploy (sam build --parallel)
                                    → INT-06 E2E Test (full flow)
                                      → INT-07 Demo Ready ☑

Week 1–2                                                  Week 10–12
```

If ANY task on this chain delays → demo delays.

PHẦN 11: SUCCESS CRITERIA

MVP Launch Criteria (trước demo)

Criteria	Status
0 compile errors trên toàn bộ Go codebase	☑ Achieved

Criteria	Status
Database size ≤ 1 MB	☑ 0.84 MB achieved
Tất cả 6 Lambda function build thành công (`sam build --parallel`)	☑
Core flow chạy end-to-end: Register → Buy ticket → Check-in	In progress
Wallet Saga: 100% correctness trên 10 test scenarios	In progress
VNPay IPN: HMAC-SHA512 verify pass trên production callback	☑
Seat allocation: không double-booking khi 10 concurrent purchases	In progress
Mobile responsive: iPhone SE → 15 Pro Max, common Android	In progress
API Swagger documentation accessible	☑ (`/swagger`)
AWS SAM deploy lên staging (ap-southeast-1)	In progress
Feature Flags: rollback bất kỳ flag trong < 30 giây	☑
X-Ray trace: cross-Lambda call graph visible trên console	☑

Post-Launch KPIs — Target Month 1

Metric	Target
API response time p95	< 500 ms
Lambda cold start (Go arm64)	< 100 ms
Ticket purchase success rate	≥ 98%
Saga compensation accuracy	100% — no lost funds
Wallet refund rate	≤ 1%
System uptime	≥ 99.5%
Events completed (→ check-in done) / created	≥ 80%
Avg time buy ticket → nhận QR email	< 3 phút
Repeat usage (same organizer > 1 event)	≥ 40%

PHẦN 12: CONVENTIONS

```
GIT:
  Branch: feature/{task-id}-short-name
          fix/{issue-id}-short-description
  PR vào develop · Squash merge
  BE-L review all PRs
  develop → main khi stable → auto-deploy via GitHub Actions → ECR → Lambda

API:
  RESTful · JSON · camelCase properties
```

```
    Error: { "error": "message", "code": "ERROR_CODE", "requestId": "uuid" }
    HTTP 400 validation · 401 auth · 403 forbidden · 409 conflict · 500 internal
    Internal endpoints: header X-Internal-Call: "true" bắt buộc

  CODE (Go):
    Package tên lowercase, single word
    Interface prefix không dùng "I" → IUserRepo ❌ · UserRepository ☑
    Max 30 lines per function — tách helper nếu dài hơn
    Comment lý do (WHY) thay vì mô tả code (WHAT)
    Error wrap: fmt.Errorf("functionName: %w", err) · Không ignore errors
    Logger: log.WithField("user_id", uid).Info("action description")
    Feature flag check: config.IsFeatureEnabled(config.FlagSagaEnabled)

  CODE (TypeScript):
    camelCase everywhere
    Tất cả API response có TypeScript interface trong types/
    Custom hooks tên useXxx
    Không dùng any — strict TypeScript

  SECURITY:
    Không hardcode secrets trong code hay config file
    Tất cả secrets qua SSM Parameter Store hoặc environment variable
    JWT secret NoEcho: true trong SAM parameters
    SQL: parameterized queries bắt buộc, không string concatenation
    Input validation trên tất cả public endpoints (common/validator)

  COMMUNICATION:
    Daily standup 15 phút (or async Discord)
    Blocker → ping BE-L immediately · Không giữ blocker > 2h
    PR review trong vòng 4 tiếng
    API contract thay đổi → notify FE ngay lập tức

  DEFINITION OF DONE:
    Code compiles (go build ./...) + go vet pass
    Unit tests pass (go test ./...)
    PR approved by BE-L
    sam validate --lint pass
    Merged vào develop
    Deployed to staging và smoke test pass
```

Document version: March 2026 · Prepared for OJT Technical Presentation · Architecture frozen at 95% Microservices completion.
Remaining 5%: Load testing, production secrets rotation, Cognito upgrade path, multi-AZ RDS promotion.